The $5x+1$ orbit of 7 is unbounded: a string-flow proof with Lean verification

StringFlow

August 13, 2026

Abstract

We study the accelerated $5x+1$ map $T(x) = (5x+1)/2^{v_2(5x+1)}$ and prove that the orbit of 7 is unbounded. The proof is organised around an exact string-flow accounting of the orbit: every step is represented by a word of 2-adic weights, every block satisfies an exact prefix identity, and the remaining obstruction is reduced to a corrected decisive window bound that carries genuine orbit reachability. The mathematical layer of the unified core is closed; the corresponding Lean statement `unified_core_final_no_hge` is the single remaining formalisation gap and is explicitly marked as pending in this paper.

Contents

1 Introduction

The $5x+1$ problem asks for the behaviour of the map

$$T(x) = \frac{5x+1}{2^{v_2(5x+1)}}, \tag{1}$$

where $v_2(n)$ denotes the exponent of 2 in the factorisation of a positive integer n. The map is the $5x+1$ analogue of the Collatz map: odd multipliers increase the size of the state, while the removal of powers of two may decrease it. For background on exact cycle arguments for related maps, see [?]. For surveys of the $3x+1$ problem and its generalizations, see [?, ?, ?].

This paper proves the following statement.

Theorem 1.1 (Main theorem)**.** *The accelerated $5x+1$ orbit of 7 is unbounded:*

$$\forall\, B \in \mathbb{N},\ \exists\, n \in \mathbb{N}, \quad B \le \mathrm{Orbit}_7(n),$$

where $\mathrm{Orbit}_7(0) = 7$ *and* $\mathrm{Orbit}_7(n+1) = T(\mathrm{Orbit}_7(n))$.

Remark 1.2 (Formalisation status)**.** The mathematical proof of **??** is presented in this paper. The Lean repository is the final authority. At the time of writing, the final unified-core valuation inequality `UnifiedCoreAudit.unified_core_final_no_hge` is still marked with `sorry`; this paper does *not* claim that the Lean formalisation is closed. All other statements marked "Lean: compiled" in the status tables have been checked by `lake build`.

1.1 Relation to the Collatz problem

For the map $T_3(x) = 3x + 1$ followed by division by the exact power of two, it is a famous open problem whether every positive orbit reaches 1. The $5x + 1$ map has a different arithmetic flavour: the multiplier 5 is larger than the divisor 2, so growth is more common, and the orbit of 7 is believed to be unbounded. The proof in this paper is not a probabilistic or heuristic statement. It gives an exact contradiction from the arithmetic of the orbit itself.

The $3x + 1$ problem has a long history of exact cycle arguments. For example, Simons and de Weger gave upper bounds on the number of possible m-cycles under additional hypotheses [**?**]. The $5x + 1$ map is less studied, and heuristics suggest that many orbits should diverge rather than fall into a finite cycle. Those heuristics are not used here. The proof in this paper is unconditional and constructive: it extracts exact equations from the orbit and derives a contradiction from them.

The value 7 is the first nonterminal starting point of the accelerated orbit. Its first steps are

$$7,\ 9,\ 23,\ 29,\ 73,\ 183,\ 229,\ 573,\ 1433,\ 3583,\ 4479,\ 5599,\ 6999,\ 8749,\ 21873,\ 54683,\ 34177.$$

The orbit is not monotone: the step from 54683 to 34177 has weight 3 and decreases the state. Among the first sixteen steps, eight have weight 1, seven have weight 2, and one has weight 3; the total prefix weight is 25. The proof does not rely on monotonicity.

1.2 Scope of the theorem

The theorem is a statement about one orbit. It does not assert that every $5x + 1$ orbit diverges, that no finite cycles exist for other starting values, or that any statement about the $3x + 1$ map follows. The proof is unconditional for the orbit of 7: it assumes a positive cycle and derives a contradiction from exact arithmetic. The same statement is the object `IsUnboundedOrbit 7` in the Lean repository.

1.3 What the proof does not use

The proof does not use cycle enumeration, probabilistic models, or transcendence estimates. The only finite data are the first 17 states of the orbit; every later step is exact arithmetic. Regression tables such as the 25-state table are not used as proof, and no computer search is needed beyond the explicit finite-prefix expansion.

1.4 What is new

The main novelty is the exact string-flow ledger. Instead of estimating the size of individual states, we encode the orbit as a word of 2-adic weights and use the word molecule identity

$$2^{W(w)}\,\mathrm{wordOrbit}(w, x_0) = 5^{\mathrm{len}(w)}\,x_0 + A(w)$$

as an exact equation. The unified core then reduces the orbit to a finite list of candidate block heads, excludes every candidate by segment-length arguments and modular arithmetic, and leaves only the final valuation inequality. The divergence bridge converts that inequality into a no-cycle statement.

1.5 Status of the formalisation

Layer	Mathematical status	Lean status
String-flow identities	proven	compiled
PMI and PMI-B budgets	proven	compiled
Finite prefix base	proven	compiled
Candidate parameterisation	proven	compiled
Segment-length exclusions	proven	compiled
Unified-core final inequality	proven	pending
Cycle bridge	proven	compiled (conditional)
Final divergence assembly	proven	compiled (conditional)

1.6 Proof chain in one line

```
finite base
  -> segment exclusions
  -> candidate family empty
  -> unified core
  -> corrected window bound
  -> no positive cycle
  -> unbounded orbit of 7
```

1.7 A one-page proof tour

1. **Encode the orbit.** Each step of the accelerated map receives a weight $t = v_2(5x+1)$. The first sixteen steps of the orbit of 7 are represented by the exact word

$$[2,1,2,1,1,2,1,1,1,2,2,2,2,1,1,3],$$

   and every prefix satisfies

$$2^{W_n}\,\mathrm{Orbit}_7(n) = 5^n \cdot 7 + A_n.$$

2. **Assume a cycle.** A positive cycle gives a closed word and a QB-8 split into rise steps ($t < 3$) and C3 steps ($t \geq 3$), both nonempty. This extraction is compiled.

3. **Parameterise the block head.** The cycle produces a reset block whose head has the form

$$x = 2^{e-1}g + \delta 5^{j-1}, \qquad g = \mathrm{Orbit}_7(j-1).$$

   The unified core derives exact segment equations for the d steps from g to x.

4. **Exclude every segment length.** The cases $d = 1$, $d = 2$, $d = 3$, and $d \geq 4$ are excluded by size bounds, modular arithmetic, and the finite-prefix base. Hence no candidate block head exists. These exclusions are compiled.

5. **Reduce to the terminal valuation.** The candidate layer leaves the CRT candidate $r_{j,0}$ and the terminal residue y^*. The final valuation inequality

$$v_2\big(5^{L+3}w_{\mathrm{term}} + 1\big) \leq H_s - 2$$

   is equivalent to $y^* \neq 0$. Mathematically this layer is closed; the Lean statement `unified_core_final_no_hge` is pending.

6. **Project to the decisive window.** The unified core implies the corrected decisive window bound
$$v_2\left(5^{k_0+1}s + \delta 5^j\right) \le 2j + 10,$$
while the block-layer balance gives the lower bound $2j + 11$. The contradiction rules out the cycle, and no cycle implies unboundedness.

The only pending Lean nodes in this tour are `unified_core_final_no_hge` and the final assembly `five_x_plus_one_diverges_at_7`. Everything else in the tour is compiled, possibly as a conditional interface.

1.8 Guide to the paper

Section **??** introduces the exact vocabulary. Section **??** states the unified core and its proof structure. Section **??** explains the corrected decisive window bound and the divergence bridge. Section **??** records the Lean verification setup. The appendix outlines the supplementary proof manual.

1.9 Proof architecture

The proof has three layers.

1. **String-flow accounting.** Each orbit step is assigned a word of weights $t_i = v_2(5x_i + 1)$. The exact relation
$$2^{W_j}\,\mathrm{Orbit}_7(j) = 5^j \cdot 7 + A_j$$
converts the orbit into an exact bookkeeping problem about prefixes of a word.

2. **Unified core.** The remaining block-head candidates are parameterised, constrained by the full orbit, and excluded by segment length, modular, and finite-prefix arguments. The mathematical layer of §36.30.23 is closed.

3. **Divergence bridge.** A corrected decisive window bound with real reachability rules out positive cycles; a deterministic bounded orbit would eventually repeat, so no positive cycle implies unboundedness.

1.10 Notation and conventions

All natural numbers are nonnegative. The notation $v_2(n)$ is used only for positive n. A word $w = (t_0, \ldots, t_{L-1})$ has length L and prefix weights
$$W_j = \sum_{i<j} t_i.$$
The word orbit is defined recursively by
$$\mathrm{wordOrbit}([], x) = x, \qquad \mathrm{wordOrbit}(t :: w, x) = \mathrm{wordOrbit}\left(w, \frac{5x+1}{2^t}\right),$$
whenever every intermediate division is exact. The exact vocabulary used in the Lean code is listed in **??**.

1.11 Lean correspondence

The following table records the correspondence used throughout the paper. The Lean column is the authoritative name in the repository.

Mathematical object	Lean name
$T(x) = (5x+1)/2^{v_2(5x+1)}$	`fiveXPlusOneStep`
Orbit $\mathrm{Orbit}_7(n)$	`fiveXPlusOneOrbit 7 n`
Unbounded orbit	`IsUnboundedOrbit`
Positive cycle	`OrbitCycle`
Valid word	`StringFlow.Word.wordValid`
Word orbit	`StringFlow.Word.wordOrbit`
Word molecule	`StringFlow.Word.wordA`
Total weight	`StringFlow.wordWeight`
PMI identity	`StringFlow.PMI.aTotal5`
Full orbit reachability	`S6Audit.FullOrbitFrom7`
General orbit reachability	`S6Audit.GeneralOrbitFrom7`
Reset window reachability	`S6Audit.ResetWindowReachability`

2 The string-flow framework

2.1 Words, weights, and molecules

Definition 2.1 (Valid word)**.** A word $w = (t_0, \ldots, t_{L-1})$ is *valid from* x_0 if the recursive definition of $\mathrm{wordOrbit}(w, x_0)$ is exact, i.e.

$$(5x_i + 1) \equiv 0 \pmod{2^{t_i}}$$

for every prefix state x_i.

Definition 2.2 (Prefix weight)**.** For a word $w = (t_0, \ldots, t_{L-1})$, the prefix weight after j steps is

$$W_j(w) = \sum_{i<j} t_i,$$

and the total weight is $W(w) = W_L(w)$.

Definition 2.3 (Word molecule)**.** For a word w of length L, the *word molecule* is the unique integer $A(w)$ satisfying

$$2^{W(w)}\,\mathrm{wordOrbit}(w, x_0) = 5^L\,x_0 + A(w)$$

for every initial state x_0. Equivalently,

$$A(w) = \sum_{j=0}^{L-1} 2^{W_j}\,5^{L-1-j}.$$

Equivalently, $A([]) = 0$ and $A(w{::}t) = 5\,A(w) + 2^{W(w)}$; this recursion is the definition used in Lean.

The word molecule converts orbit arithmetic into exact prefix arithmetic. It is the object that appears in the Lean statements `StringFlow.Word.wordA` and `StringFlow.PMI.aTotal5`. The formalisation is written in Lean [?] using Mathlib [?].

Theorem 2.4 (Exact word orbit identity). *For every valid word $w = (t_0, \ldots, t_{L-1})$ from x_0,*

$$2^{W(w)}\,\mathrm{wordOrbit}(w, x_0) = 5^L\,x_0 + A(w).$$

Proof sketch. Proceed by induction on the length of w. For the singleton word $[t]$,

$$2^t\,\mathrm{wordOrbit}([t], x_0) = 5x_0 + 1,$$

and $A([t]) = 1$. For $w' = w{::}t$,

$$2^{W(w')}\,\mathrm{wordOrbit}(w', x_0) = 2^{W(w)}\,(5\,\mathrm{wordOrbit}(w, x_0) + 1) = 5\,(2^{W(w)}\,\mathrm{wordOrbit}(w, x_0)) + 2^{W(w)},$$

which matches the recursive formula for $A(w')$. The identity is the exact ledger used throughout the paper. □

2.2 The valuation decomposition

Every positive integer n has a unique decomposition

$$n = 2^{v_2(n)}\,\mathrm{oddpart}(n), \qquad \mathrm{oddpart}(n) \equiv 1 \pmod 2.$$

In Lean this is recorded by `n_eq_two_pow_mul_oddPart` and `oddPart_odd_of_pos`. The exact step weight $t = v_2(5x+1)$ is the largest weight for which $(5x+1)/2^t$ is an integer. Word validity only asks for divisibility, so a legal word may use any $t \le v_2(5x+1)$. This is the precise sense in which GeneralOrbitFrom7 is weaker than FullOrbitFrom7.

2.3 Valuation toolkit

The following elementary facts are used throughout without comment:

- $n = 2^{v_2(n)}\,\mathrm{oddpart}(n)$ with $\mathrm{oddpart}(n)$ odd;
- if u is odd, then $v_2\big(2^k u\big) = k$;
- for odd x, $v_2(5x+1) \ge 1$;
- $T(x) \le (5x+1)/2$;
- a C3 step from $x \ge 1$ satisfies $(5x+1)/2^t < x$.

These elementary facts are formalised in Lean: the odd-part decomposition, the valuation rules, the step bound, and the C3 decrease lemma.

Proposition 2.5 (Exact orbit bound). *For every $n \ge 2$,*

$$\mathrm{Orbit}_7(n) < 5^n.$$

This is proved in Lean (`CycleBridge.fiveXPlusOneOrbit_lt_five_pow`).

Proof. For $n = 2$ the claim is checked directly. If $x = \mathrm{Orbit}_7(k) < 5^k$, then

$$5x + 1 < 5^{k+1} + 1 < 2 \cdot 5^{k+1},$$

so $(5x+1)/2 < 5^{k+1}$. The accelerated step satisfies $T(x) \le (5x+1)/2$, hence $\mathrm{Orbit}_7(k+1) < 5^{k+1}$. □

2.4 The first steps

The first 16 states and step weights make the definitions concrete:

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\mathrm{Orbit}_7(n)$	7	9	23	29	73	183	229	573	1433	3583	4479	5599	6999	8749	21873	54683
t_n	2	1	2	1	1	2	1	1	1	2	2	2	2	1	1	3

The step from depth 15 to depth 16 has weight 3, and the full orbit then reaches 34177. The table is proved by explicit rewriting in `FinitePrefix.lean`.

Example 2.6. The word $w = [2, 1]$ is valid from 7:

$$7 \xrightarrow{t=2} 9 \xrightarrow{t=1} 23,$$

so $\mathrm{wordOrbit}(w, 7) = 23$. Its prefix weights are $W_0 = 0$, $W_1 = 2$, $W = 3$, and

$$A(w) = 2^0 \cdot 5^1 + 2^2 \cdot 5^0 = 9.$$

The exact word-orbit identity gives

$$2^3 \cdot 23 = 184 = 5^2 \cdot 7 + 9.$$

The word $[1, 2]$ is valid from 9:

$$9 \xrightarrow{t=1} 23 \xrightarrow{t=2} 29, \qquad A([1, 2]) = 2^0 \cdot 5^1 + 2^1 \cdot 5^0 = 7,$$

and $2^3 \cdot 29 = 232 = 5^2 \cdot 9 + 7$.

Validity is weaker than tracking the accelerated orbit: the word $[1, 1]$ is valid from 9 in the divisibility sense, because $46/2 = 23$ and $116/2 = 58$ are integers, but the exact second weight at 23 is $v_2(116) = 2$, not 1. Hence $\mathrm{wordOrbit}([1, 1], 9) = 58$ is an intermediate state not on the accelerated orbit of 7. This is precisely the distinction between GeneralOrbitFrom7 and FullOrbitFrom7 used in **??**.

2.5 The prefix ledger

Let W_n be the prefix weight after n exact steps from 7, and let A_n be the word molecule of that exact prefix word. The first values are:

n	t_n	W_n	A_n
0	–	0	0
1	2	2	1
2	1	3	9
3	2	5	53
4	1	6	297
5	1	7	1549
6	2	9	7873
7	1	10	39877
8	1	11	200409
9	1	12	1004093
10	2	14	5024561
11	2	16	25139189
12	2	18	125761481
13	2	20	629069549
14	1	21	3146396321
15	1	22	15734078757
16	3	25	78674588089

Each row is an instance of the exact word-orbit identity:

$$2^{W_n}\,\mathrm{Orbit}_7(n) = 5^n \cdot 7 + A_n.$$

For example, $2^{25} \cdot 34177 = 5^{16} \cdot 7 + 78674588089$. The column A_n is the word molecule of the exact prefix word and is the value of `StringFlow.Word.wordA` on that prefix.

2.6 The PMI identity

Theorem 2.7 (PMI prefix identity)**.** *Let w be a valid word of length L from x_0, let W_j be its prefix weights, and define the margin*

$$m_j = j \log_2 5 - W_j.$$

Then

$$\sum_{j=0}^{L-1} 2^{-m_j} = \frac{5\,A(w)}{5^L}.$$

The cleared-denominator form

$$\sum_{j=0}^{L-1} 2^{W_j}\,5^{L-j} = 5\,A(w)$$

is the identity $\mathrm{aTotal5} = 5\,\mathrm{aTotal}$ formalised in `Pmi.lean`. For a closed cycle word of period P and cycle state m, substituting the cycle equation gives

$$\mathrm{aTotal5}(P,t) = 5m(2^T - 5^P), \qquad T = W(w).$$

Derivation sketch. Write

$$A(w) = \sum_{j=0}^{L-1} 2^{W_j}\,5^{L-1-j}.$$

Then

$$\frac{5\,A(w)}{5^L} = \sum_{j=0}^{L-1} 2^{W_j}\,5^{-j}.$$

Since $2^{-m_j} = 2^{W_j}5^{-j}$, the identity follows. The calculation is a finite rational identity; no limits or estimates are involved. □

2.7 Prefix budgets and PMI-B

The PMI-B layer counts the positions in a word that violate the prefix budget. For a closed cycle word of period P, the total weight satisfies

$$2^{W(w)}\,m = 5^P\,m + A(w)$$

for the cycle state m, so $W(w) > P\log_2 5$. The PMI-B inequalities then compare the number of "bad" prefixes with the available budget. The corresponding PMI-B no-bad-prefix theorem and the margin-balance theorem are compiled in Lean.

Lemma 2.8 (PMI-B bad-prefix count)**.** *For a closed cycle word of period P, the number of bad prefixes and the total weight satisfy the PMI-B bounds*

$$\mathrm{bad}(w) \le \frac{2^{W(w)}}{5^P} A(w),$$

and every true prefix of a small-budget word satisfies the margin balance inequality. The counting bound and the no-bad-prefix conclusion are compiled in Lean.

Lemma 2.9 (Margin balance)**.** *Let $\alpha = \log_2 5$, and let C_1, C_2, C_3 be the number of steps of weight 1, weight 2, and weight at least 3 in a closed cycle word. Then the nonnegative-margin prefix condition implies*

$$(3-\alpha)C_3 \le (\alpha-1)C_1 + (\alpha-2)C_2,$$

and the strict version replaces $\le$ by $<$. The margin-balance inequalities are compiled in Lean.

2.8 PMI-B derivation

Write $B = \mathrm{bad}(w)$ for the number of bad proper prefixes of a closed cycle word of period P, and let T be its total weight. The $j = 0$ term of aTotal5 contributes 5^P, and every bad prefix j contributes at least another 5^P:

$$5^P + B \cdot 5^P \le \mathrm{aTotal5}(P, t).$$

Together with the cycle equation

$$\mathrm{aTotal5}(P, t) = 5m(2^T - 5^P),$$

this is the exact counting bound

$$5^P + B \cdot 5^P \le 5m(2^T - 5^P).$$

Corollary 2.10 (Small budget, no bad prefix)**.** *If $5m(2^T - 5^P) < 2 \cdot 5^P$, then*

$$2^{W_j} < 5^j \qquad (1 \le j < P).$$

Proof. The counting bound gives $B \cdot 5^P < 5^P$, hence $B = 0$. □

The statements are `pmi_b_count` and `pmi_b_no_bad_prefix` in `Pmi.lean`.

2.9 PMI-B worked example

The word $[3, 1]$ has prefix weights

$$W_0 = 0, \quad W_1 = 3, \quad W_2 = 4,$$

and molecule $A([3, 1]) = 13$. Its margins are

$$m_0 = 0, \qquad m_1 = \log_2 5 - 3 < 0, \qquad m_2 = 2\log_2 5 - 4 > 0,$$

so the proper prefix of length 1 is bad and $B = 1$. The cleared PMI sum is

$$\mathrm{aTotal5}(2, t) = 5^2 + 2^3 \cdot 5^1 = 25 + 40 = 65,$$

and the counting bound reads

$$5^2 + B \cdot 5^2 = 25 + 25 = 50 < 65.$$

The exact orbit word $[2, 1, 2]$ from 7, by contrast, has all margins positive. This is the exact arithmetic that the PMI-B layer counts: one bad prefix costs one extra 5^P, and no estimate replaces the count.

2.10 Rise steps and residues modulo 5

A rise step of weight 1 satisfies $2y = 5x + 1$, hence $y \equiv 3 \pmod 5$. A rise step of weight 2 satisfies $4y = 5x + 1$, hence $y \equiv 4 \pmod 5$. These two residue rules are the only constraints that a rise step imposes on the next state modulo 5; they are formalised in `rise_step_next_mod_five` and are used by the cycle-bridge block-head residue lemma.

2.11 Block equations

A block is a contiguous segment of the orbit. For a block starting at state r_0 with step weights $u_1, \ldots, u_L$, the block equation is

$$2^U r_L = 5^L r_0 + B,$$

where $U = \sum u_i$ and B is the block molecule. The reset block of weight $t \in \{1, 2\}$ and parameter $\delta \in \{1, 3\}$ satisfies

$$2^t(r_j + 1) = 5^{k_0+1}s + \delta 5^j$$

in the $t = 2$ case, and the corresponding $t = 1$ equation

$$2(r_j + 1) = 5^{k_0+1}s + 5^j - 2.$$

These are the equations encoded in `S6Audit.ResetHeadEq` and used by the corrected window bound.

Example 2.11 (Block equation)**.** The block starting at 29 with weights $[1, 1, 2]$ has $L = 3$, $U = 4$, and block molecule $B = 39$:

$$2^4 \cdot 229 = 3664 = 5^3 \cdot 29 + 39.$$

2.12 Lean correspondence

Mathematical object	Lean name	Status
Word validity	`Word.wordValid`	compiled
Word orbit	`Word.wordOrbit`	compiled
Word molecule	`Word.wordA`	compiled
Prefix weight	`wordWeight`	compiled
PMI identity	`Pmi.aTotal5`	compiled
PMI-B budget	`cycleWord_pmi_b_count`	compiled
Exact orbit bound	`fiveXPlusOneOrbit_lt_five_pow`	compiled
Full orbit reachability	`FullOrbitFrom7`	compiled
General orbit reachability	`GeneralOrbitFrom7`	compiled
Reset window reachability	`ResetWindowReachability`	compiled

2.13 Reachability: full orbit versus general orbit

Definition 2.12 (Full and general orbit reachability)**.** Let FullOrbitFrom7(x) mean that x lies on the exact accelerated orbit of 7:

$$\text{FullOrbitFrom7}(x) \iff \exists n,\ \text{Orbit}_7(n) = x.$$

Let GeneralOrbitFrom7(x) mean that x is reachable from 7 by some finite word whose divisions are exact, without any restriction on the weights.

The difference is essential. The general orbit permits submaximal weights and therefore intermediate states that are not states of the exact accelerated orbit; the full orbit does not. The 25-state table used in earlier regression work is a finite check and is not used as a substitute for the full-orbit exclusion.

2.14 Regression versus proof

Regression data	Proof requirement
25-state table	17-state exact expansion
simulated orbit values	explicit rewriting, no `native_decide`
statistical weight profile	exact word ledger with valuation identities
heuristic unboundedness	contradiction from a cycle assumption

Regression tables are used in this paper only to locate candidates; they never enter a proof step. The 25-state table, for example, can confirm that no early state repeats, but it cannot rule out a later cycle. Every proof step instead uses the exact word-orbit identity, the explicit finite prefix, or a compiled Lean statement. This distinction is the reason the paper separates FullOrbitFrom7 from GeneralOrbitFrom7 and keeps the 17-state expansion explicit.

Definition 2.13 (Reset window reachability)**.** The predicate $\mathrm{ResetWindowReachability}(j, k_0, t, \delta, s)$ asserts the existence of r and r_j such that

$$k_0 + 1 \le j, \qquad s5^{k_0} = r + 1, \qquad \mathrm{GeneralOrbitFrom7}(r), \qquad \mathrm{FullOrbitFrom7}(r_j),$$

and the exact reset equation $\mathrm{ResetHeadEq}(s, j, k_0, t, \delta, r_j)$ holds, together with the size bound $s < 5^{j-1-k_0}$, the oddness of s and r_j, and $5 \nmid s$.

The predicate is the reachability input of the corrected window bound. It fixes the same j, k_0, t as the window under consideration; a weaker predicate that only fixes the difference $j - k_0 - 1$ would allow the reset witness to use different indices.

2.15 Notation discipline

The step weight at depth j is $t_j = v_2(5\mathrm{Orbit}_7(j) + 1)$. The prefix weight before that step is W_{j-1}. These two quantities are not interchangeable: t_j is the cost of the next step, while W_{j-1} is the accumulated cost before it. The proof never substitutes one for the other.

2.16 Glossary

Term	Meaning
word	finite list of step weights
valid word	word whose orbit divisions are exact
prefix weight	accumulated weight before a step
word molecule	exact prefix sum of a word
margin	$j \log_2 5 - W_j$
block	contiguous orbit segment
rise step	step of weight 1 or 2
C3 step	step of weight at least 3
reset window	reachable block with exact reset equation
failure window	reset block whose valuation contradicts the bound
full orbit	exact accelerated orbit of 7
general orbit	orbit with arbitrary legal weights

2.17 Variable dictionary

Symbol	Meaning
j	reset step depth
k_0	exponent in $s5^{k_0} = r + 1$
t	reset step weight, $t \in \{1, 2\}$
δ	reset parameter: 1 for $t = 1$, $\{1, 3\}$ for $t = 2$
s	odd terminal part with $5 \nmid s$
e	incoming step weight, $e \geq 2$
g	full-orbit state $\mathrm{Orbit}_7(j-1)$
x	candidate predecessor $2^{e-1}g + \delta 5^{j-1}$
r_j	block head after the reset step
r_s	terminal state of the block tail
t_j	step weight at depth j
W_j	prefix weight at depth j
L	block-tail length
H_s	unified-core threshold parameter

2.18 Why loose estimates do not close the problem

Heuristic models predict roughly logarithmic growth, but the orbit contains long stretches of weight-2 steps that consume 2-adic value, interspersed with weight-1 and weight-≥ 3 steps. Bounds that only control the average growth are too weak: the decisive obstruction is an exact inequality with constants 13 and +12, and any weakening of these constants destroys the contradiction. Probabilistic equidistribution statements and transcendence estimates do not provide the exact 2-adic cancellations used in the unified core.

A heuristic word with the same average weight can fail the exact valuation identity by a single unit of 2-adic cancellation. The arithmetic counterexample in the divergence bridge is exactly such a witness: it satisfies every size, parity, and coprimality condition but is not orbit-reachable. The proof therefore keeps every divisibility condition exact.

3 The unified core

3.1 Block-head candidate parameterisation

Let

$$r = \frac{A_j + 5^j q}{2^{W_j}}$$

be a block-head state. The unified core writes the predecessor candidate in the form

$$x = 2^{e-1}g + \delta\, 5^{j-1}, \qquad g = \mathrm{Orbit}_7(j-1),$$

where $e \geq 2$ is the incoming step weight, $\delta \in \{1, 3\}$ is the reset parameter, and the reset step has weight $t \in \{1, 2\}$. The exact relations are:

1. the reset equation $\mathrm{ResetWindowReachability}(j, k_0, t, \delta, s)$ fixes s, j, k_0, t, δ;
2. the first-block terminal equation gives $s - 1 = 2^{e-1}g$;
3. the candidate satisfies $x = 2^{e-1}g + \delta 5^{j-1}$.

These statements are formalised in the Lean bridge module `UnifiedCoreBridge.lean`: the reset-predecessor lemma, the candidate parameterisation, and the first-block terminal identity.

3.2 Segment-length exclusions

Lemma 3.1 (Segment-length exclusions)**.** *Let d be the length of the full-orbit segment from g to the candidate x. Then $d \neq 1$, $d \neq 2$, $d \neq 3$, and $d \geq 4$ is impossible. Hence the candidate family is empty.*

Remark 3.2 (Status of **??**)**.** The mathematical proof is closed. The Lean side has compiled the $d = 1$, $d = 2$, $d = 3$, and $d \geq 4$ exclusions, together with the $e = 3$, $j = 17$ modular exclusions.

Statement	Math	Lean
Candidate parameterisation	proven	compiled
Full-orbit to candidate derivation	proven	pending
Reset-head predecessor	proven	compiled
First-block terminal equation	proven	compiled
$d = 1$ exclusion	proven	compiled
$d = 2$ exclusion	proven	compiled
$d = 3$ family exclusion	proven	compiled
$d \geq 4$ exclusion	proven	compiled
Unified-core final inequality	proven	pending

3.3 Word rigidity and the first block

The full-orbit reachability of the block head forces the first block to have the rigid shape

$$y \longrightarrow x \longrightarrow r_j \longrightarrow z \longrightarrow w,$$

with $k_0 = 0$ and the first-block terminal equation

$$s - 1 = 2^{e-1} g.$$

The Lean side has compiled the first-block terminal identity and the candidate parameterisation. The complete derivation from the block-head premises and FullOrbitFrom7(r) to this candidate parameterisation is mathematically closed but is still marked as Lean pending.

3.4 The q0 interval and block-head identity

The block-head numerator satisfies the exact identity

$$A_j + 5^j q = 2^L \left(B + \delta 5^j\right), \qquad 0 < B < 5^j, \quad A_j < 5^j,$$

which forces

$$q = m + \delta\, 2^L, \qquad m < 2^L.$$

This is the explicit q_0 form. Under the reset equation, the same numerator has the block-head form

$$A_j + 5^j q = 2^{W_p} \left(5^{k+1} s - 4 + \delta 5^j\right),$$

which is the block-head identity of a reset. The interval condition

$$2^{W_p} \leq q < 2^{W_j}$$

is equivalent to the block-head bounds

$$5^j \leq 2^t r_j, \qquad r_j < 5^j,$$

by the q_0-interval criterion. These three statements are compiled.

3.5 Segment equations

Let $u_1, \ldots, u_d$ be the step weights of the full-orbit segment from $g = \mathrm{Orbit}_7(j-1)$ to $x = \mathrm{Orbit}_7(j-1+d)$, and let $W = u_1 + \cdots + u_d$. The exact segment equation is

$$2^{W+e-1}g + 2^W \delta 5^{j-1} = 5^d\, g + \sum_{i=0}^{d-1} 2^{W_i} 5^{d-1-i},$$

where W_i is the prefix weight inside the segment, $W = W_d$, and e is the incoming step weight. This is the candidate form of the Lean equation

$$2^W\, x = 5^d\, g + A(w)$$

obtained by substituting $x = 2^{e-1}g + \delta 5^{j-1}$.

The derivation is direct: the segment word w has length d and maps g to x, so the exact word-orbit identity gives $2^W x = 5^d g + A(w)$. Substitution of the candidate parameterisation is exactly the segment-candidate equation formalised in Lean. In the special cases $d = 1$ and $d = 2$, the word molecules are $A = 1$ and $A = 5 + 2^{u_1}$.

For $d = 1$ the equation reduces to

$$5g + 1 = 2^{1+4a}x,$$

which is impossible by the size bound $g < 5^{j-1}/4$. For $d = 2$ the surviving branch forces the two modular equations

$$5^{j-1} \equiv 6 \pmod{17}, \qquad 5^{j-1} \equiv 45 \pmod{256},$$

which contradict each other modulo 16. For $d = 3$ the only surviving family has first big step at depth $j + 4$ with weight 6, contradicting the finite base. For $d \geq 4$, the first big step is either at depth $j - 2$ with weight 3, or later with weight at least 5, again contradicting the finite base.

The orbit-level $d = 1$ and $d = 2$ exclusions are compiled in `UnifiedCoreBridge.lean`.

3.6 Detailed exclusion arguments

$d = 1$. The segment equation is

$$5g + 1 = 2^{1+4a}\,(2^{e-1}g + \delta 5^{j-1}).$$

For $e \geq 3$ or $a \geq 1$, the coefficient 2^{e+4a} is at least 8, while the right-hand side contains a positive multiple of $2^{1+4a}\delta 5^{j-1}$, forcing a contradiction with the size bound $g < 5^{j-1}/4$. The only remaining case is $e = 2, a = 0$, which gives

$$g + 1 = 2\delta\, 5^{j-1},$$

so $g \geq 2 \cdot 5^{j-1} - 1$ for $\delta \geq 1$, again contradicting $g < 5^{j-1}/4$.

$d = 2$. The size constraint $2^{1+4a}\delta \leq 24$ forces $a = 0$. The segment equation is

$$(2^{u_1+e} - 25)g + 2^{u_1+1}\delta 5^{j-1} = 5 + 2^{u_1}.$$

The only surviving branch is $\delta = 1$, $e = 2$, $u_1 = 1$; substituting into the segment equation gives

$$(8 - 25)g + 4 \cdot 5^{j-1} = 7,$$

hence

$$17g = 4 \cdot 5^{j-1} - 7,$$

hence $j - 1 \equiv 3 \pmod{16}$. The candidate congruence $x \equiv 743 \pmod{1280}$ then forces

$$5^{j-1} \equiv 45 \pmod{256},$$

so $j - 1 \equiv 7 \pmod{64}$, contradicting $j - 1 \equiv 3 \pmod{16}$.

3.7 The $d = 2$ survivor in detail

The survivor equations are

$$17g = 4 \cdot 5^{j-1} - 7, \qquad x \equiv 743 \pmod{1280}, \qquad x = 2g + 5^{j-1}.$$

Modulo 17, the first equation gives $4 \cdot 5^{j-1} \equiv 7$, so

$$5^{j-1} \equiv 6 \pmod{17}.$$

Since $5^3 \equiv 6 \pmod{17}$ and the order of 5 modulo 17 is 16, we get $j - 1 \equiv 3 \pmod{16}$.

Eliminating g gives

$$17x = 25 \cdot 5^{j-1} - 14,$$

so $x \equiv 743 \pmod{1280}$ forces

$$25 \cdot 5^{j-1} \equiv 1125 \pmod{1280}.$$

Dividing by 5 and multiplying by the inverse of 5 modulo 256 reduces this to

$$5^{j-1} \equiv 45 \pmod{256}.$$

Since $5^7 \equiv 45 \pmod{256}$ and the order of 5 modulo 256 is 64, we get $j - 1 \equiv 7 \pmod{64}$. The two congruences contradict each other modulo 16.

$d = 3$. The remaining family is

$$t_j = 2, \quad \delta = 1, \quad e = 2, \quad u_1 = u_2 = 1, \quad a = 0,$$

so $u_3 = 1$; the segment equation becomes

$$16g + 8 \cdot 5^{j-1} = 125g + 39,$$

which gives

$$g = \frac{8 \cdot 5^{j-1} - 39}{109}, \qquad j - 1 \equiv 923 \pmod{1728}.$$

The word shape is

$$g \to x \to r_j \to z \to w,$$

and the first step of weight at least 3 is the final step at depth $j + 4$ with weight 6. The finite base gives first big weight 3 at depth 15, a contradiction.

$d \geq 4$. The first big step is one of three types. If $e \geq 3$, the first big step is $g_{\text{prev}} \to g$; the finite base forces $e = 3$ and $j = 17$, and the candidate residue modulo 640 or 1280 is excluded. If $e = 2$ and $a \geq 1$, the first big step is $y \to x$ with weight $1 + 4a \geq 5$. If $e = 2$ and $a = 0$, the first big step is $z \to w$ with weight 5 or 6. All three contradict the finite base.

3.8 Branch ledger

Case	Surviving branch	Contradiction
$d = 1$	$e = 2$, $a = 0$	$g = 2 \cdot 5^{j-1} - 1$ vs. $g < 5^{j-1}/4$
$d = 2$	$\delta = 1$, $e = 2$, $u_1 = 1$	$j - 1 \equiv 3 \pmod{16}$ and $j - 1 \equiv 7 \pmod{64}$
$d = 3$	$t_j = 2$, $\delta = 1$, $e = 2$, $u_1 = u_2 = 1$, $a = 0$	first big step at depth $j+4$ has weight 6, not 3
$d \geq 4$	three big-step types	weight ≥ 5, or residue exclusion for $e = 3$, $j = 17$

Each row records the exact contradiction used in the mathematical proof. The Lean names for the individual exclusions are listed in the status tables above.

3.9 Modular arithmetic ledger

Case	Congruence and role	Lean
$d = 1$	no congruence; the size bound $g < 5^{j-1}/4$ suffices	`d1_exclusion`
$d = 2$	$5^{j-1} \equiv 6 \pmod{17}$ and $x \equiv 743 \pmod{1280}$ force $j - 1 \equiv 3 \pmod{16}$ and $5^{j-1} \equiv 45 \pmod{256}$	`d2_exclusion`
$d = 3$	$j - 1 \equiv 923 \pmod{1728}$ fixes the unique surviving family	`d3_family_big_weight_excluded`
$d \geq 4$	the $e = 3$, $j = 17$ branch is excluded by residues modulo 640 or 1280	`dge4_e3_j17_t1_excluded`, `dge4_e3_j17_t2_delta3_excluded`

These congruences are exact and are the only modular inputs needed after the size estimates; they are proved inside the corresponding segment exclusions.

3.10 Worked modular ledger

The three congruence families used by the exclusions are:

$$\begin{aligned} 5^{j-1} &\equiv 6 \pmod{17} && \Longrightarrow j - 1 \equiv 3 \pmod{16}, \\ 5^{j-1} &\equiv 45 \pmod{256} && \Longrightarrow j - 1 \equiv 7 \pmod{64}, \\ 5^{j-1} &\equiv 73 \pmod{109} && \text{for the } d = 3 \text{ family.} \end{aligned}$$

The first two follow from $5^3 \equiv 6 \pmod{17}$ and $5^7 \equiv 45 \pmod{256}$ with the respective orders 16 and 64. The third is the integrality condition for

$$g = \frac{8 \cdot 5^{j-1} - 39}{109}.$$

Its consistency is checked by repeated squaring modulo 109:

k	$5^k \bmod 109$	k	$5^k \bmod 109$
1	5	64	97
2	25	128	35
4	80	256	26
8	78	512	22
16	89	923	73
32	73		

The last entry uses

$$923 = 512 + 256 + 128 + 16 + 8 + 2 + 1$$

and the corresponding product of residues, giving

$$5^{923} \equiv 73 \pmod{109}, \qquad 8 \cdot 73 = 584 \equiv 39 \pmod{109}.$$

These checks are part of the compiled exclusions; the paper records them here so that the modular layer can be read without a calculator.

3.11 Block-head candidate exclusion

The segment-length exclusions show that no candidate

$$x = 2^{e-1}g + \delta 5^{j-1}$$

can satisfy the full set of constraints coming from `All36_20PremisesNoHge` and FullOrbitFrom7(r). Hence no block head satisfies the candidate family of document 36.30.7. This is the mathematical closure of the unified core's candidate layer. The Lean side has compiled the generic exclusion theorems, while the complete derivation from the no-$H_{\geq}$ premises to those exclusions remains pending.

3.12 Proof outline

1. Extract the block head $r = (A_j + 5^j q)/2^{W_j}$ from the no-$H_{\geq}$ premises.
2. Use FullOrbitFrom7(r) to place the block head on the exact orbit of 7 and obtain the incoming weight e and the state $g = \mathrm{Orbit}_7(j-1)$.
3. Parameterise the predecessor by $x = 2^{e-1}g + \delta 5^{j-1}$.
4. Write the exact segment equation for d steps from g to x.
5. Exclude $d = 1$, $d = 2$, $d = 3$, and $d \geq 4$; hence the candidate family is empty.
6. Pass through the CRT candidate $r_{j,0}$ and the residue y^* to obtain the final valuation inequality.

3.13 From candidate emptiness to the final inequality

The exclusion of every candidate block head leaves the CRT candidate $r_{j,0}$ and the terminal residue y^*. The terminal inequality

$$v_2\big(5^{L+3} w_{\mathrm{term}} + 1\big) \leq H_s - 2$$

is equivalent to $y^* \neq 0$, and the remaining Lean statement `unified_core_final_no_hge` is exactly this valuation inequality under the no-$H_{\geq}$ premises and the full-orbit reachability of the block head. The mathematical layer is closed; the Lean statement is pending.

3.14 The remaining Lean statement

The exact statement of the pending theorem is:

```
theorem unified_core_final_no_hge
    (j Wp Wj q Aj A_s s W_s r_s L H_s : Nat)
    (weight : Nat -> Nat) (r : Nat)
    (hPrem : All36_20PremisesNoHge
       j Wp Wj q Aj A_s s W_s r_s L H_s weight)
    (hrj : r = (Aj + 5 ^ j * q) / 2 ^ Wj)
    (hReach : S6Audit.FullOrbitFrom7 r)
    (hH : 2 <= H_s) :
    twoValuation (5 ^ (L + 3) * wTerminal L r_s + 1) <= H_s - 2 := by
  sorry
```

The premises are: the no-$H_{\geq}$ record, the block-head equality $r = (A_j + 5^j q)/2^{W_j}$, the full-orbit reachability of r, and the domain condition $2 \leq H_s$. The conclusion is the terminal valuation inequality of `??`. This is the single `sorry` remaining in the unified-core audit file.

3.15 The CRT candidate and the terminal residue

Let $n = s - j$ and $\Delta = W_s - W_j$. The block molecule B of the suffix satisfies

$$3B + 2^{\Delta} \leq 3 \cdot 5^n - 2 \cdot 4^n,$$

and the CRT candidate $r_{j,0}$ is the least nonnegative solution of the exact integer equation

$$3 \cdot 5^n \, r_{j,0} + 3B + 2^{\Delta} = 2^{\Delta+L+4} \left(u + m' \, 2^{H_s - 1}\right),$$

where u is the least nonnegative residue of $-5^{-(L+3)}$ modulo 2^{H_s-1} and $m' \geq 0$.

The terminal odd part is

$$w_{\mathrm{term}} = \frac{3r_s + 1}{2^{L+4}},$$

and the residue

$$y^* = -\left(w_{\mathrm{term}} + 5^{-(L+3)}\right)(3 \cdot 5^s)^{-1} \pmod{2^{H_s-1}}$$

is the obstruction to the final valuation inequality. The Lean equivalences

$$v_2\big(5^{L+3} w_{\mathrm{term}} + 1\big) \leq H_s - 2 \iff y^* \neq 0$$

are proved in `terminal_bound_iff_yStar_ne_zero`. The remaining Lean statement `unified_core_final_no_hge` is the passage from the no-$H_{\geq}$ premises and FullOrbitFrom7(r) to this inequality.

3.16 Derivation of the CRT equation

The CRT equation is not introduced as an independent congruence; it is the failure equation of the terminal valuation written in block coordinates. Let $n = s - j$ and $\Delta = W_s - W_j$, and let B be the block molecule of the suffix of length L. The suffix is a valid word from $r_{j,0}$ to r_s, so the exact word-orbit identity gives

$$2^{\Delta} \, r_s = 5^n \, r_{j,0} + B.$$

The terminal odd part is defined by

$$w_{\mathrm{term}} = \frac{3r_s + 1}{2^{L+4}},$$

so, multiplying the tail equation by $3 \cdot 2^{\Delta}$ and adding 2^{Δ},

$$2^{\Delta+L+4}\, w_{\mathrm{term}} = 2^{\Delta}(3r_s + 1) = 3 \cdot 5^n\, r_{j,0} + 3B + 2^{\Delta}.$$

This single identity is the bridge between the orbit equation and the valuation inequality.

The valuation inequality

$$v_2\big(5^{L+3} w_{\mathrm{term}} + 1\big) \le H_s - 2$$

fails exactly when

$$5^{L+3} w_{\mathrm{term}} + 1 \equiv 0 \pmod{2^{H_s-1}},$$

i.e. exactly when

$$w_{\mathrm{term}} \equiv -5^{-(L+3)} \pmod{2^{H_s-1}}.$$

The inverse exists because 5 is odd. Let u be the least nonnegative residue of $-5^{-(L+3)}$ modulo 2^{H_s-1}. Substituting the failure congruence into the identity above gives the CRT equation

$$3 \cdot 5^n\, r_{j,0} + 3B + 2^{\Delta} = 2^{\Delta+L+4}\left(u + m'\, 2^{H_s-1}\right)$$

for some integer $m' \ge 0$. This is the exact form used in `rj0_spec_eq`.

The terminal residue y^* is the normalised failure residue

$$y^* = -\left(w_{\mathrm{term}} + 5^{-(L+3)}\right)(3 \cdot 5^s)^{-1} \pmod{2^{H_s-1}}.$$

By construction $y^* = 0$ if and only if the failure congruence holds, so the valuation inequality is equivalent to $y^* \neq 0$. The factor $(3 \cdot 5^s)^{-1}$ is the normalisation inherited from the orbit equation for the terminal state; it is invertible modulo 2^{H_s-1} because both 3 and 5^s are odd. The equivalence is `terminal_bound_iff_yStar_ne_zero`.

3.17 Size conditions

The sufficient direction uses two inverse-size conditions. The first is the block-tail bound

$$3B + 2^{\Delta} \le 3 \cdot 5^n - 2 \cdot 4^n,$$

and the second is the base-size condition

$$3 \cdot 5^s + (3 \cdot 5^n - 2 \cdot 4^n) \le 2^{\Delta+L+4}\, u,$$

which together imply $r_{j,0} \ge 5^j$. This is proved in Lean by `rj0_ge_of_size_conditions_no_hge`. The full equivalence through the CRT lift remains part of the unified core; it is not claimed as an independent step.

3.18 Why the size conditions imply $r_{j,0} \geq 5^j$

The two size conditions are exactly what the CRT equation needs in order to lift the candidate above 5^j. The first condition

$$3B + 2^{\Delta} \leq 3 \cdot 5^n - 2 \cdot 4^n$$

controls the correction term, and the second condition

$$3 \cdot 5^s + (3 \cdot 5^n - 2 \cdot 4^n) \leq 2^{\Delta+L+4}\, u$$

controls the leading term. From the CRT equation,

$$3 \cdot 5^n\, r_{j,0} = 2^{\Delta+L+4} \left(u + m' 2^{H_s - 1}\right) - 3B - 2^{\Delta}.$$

The right-hand side is at least

$$2^{\Delta+L+4}\, u - (3 \cdot 5^n - 2 \cdot 4^n),$$

and the second condition bounds this below by $3 \cdot 5^s$. Therefore

$$r_{j,0} \geq \frac{3 \cdot 5^s}{3 \cdot 5^n} = 5^{s-n} = 5^j,$$

because $n = s - j$. This is the arithmetic content of `rj0_ge_of_size_conditions_no_hge`; the Lean statement packages the divisibility and nonnegativity facts that the paper suppresses.

The lower bound $r_{j,0} \geq 5^j$ is then compared with the interval constraints carried by the no-$H_{\geq}$ premises. The passage from the block-head interval to the valuation inequality is exactly the remaining Lean bridge `unified_core_final_no_hge`; the paper does not claim it as closed.

3.19 Relation to source documents

Document	Paper	Lean
36.30.22	candidate parameterisation	`UnifiedCoreBridge` compiled
36.30.23	segment exclusions	`FinitePrefix` and `UnifiedCoreBridge`; compiled
36.30.7	block-head candidate exclusion	pending full derivation
36.20	final valuation inequality	`UnifiedCoreAudit` pending

3.20 The finite base

The full orbit is explicitly expanded for the first 17 states:

$$7, 9, 23, 29, 73, 183, 229, 573, 1433, 3583, 4479, 5599, 6999, 8749, 21873, 54683, 34177.$$

The first step of weight ≥ 3 is the step from depth 15 to depth 16, and its weight is exactly 3. This is proved in `FinitePrefix.lean` by explicit rewriting, with no `native_decide`.

The expansion stops at 17 states because every exclusion either uses the first big step (depth 15, weight 3) or reduces the branch to $j = 17$. No later full-orbit data is needed.

3.21 How the finite base is used

Finite fact	Lean name	Used in
first 17 states expanded	`fullOrbitIter_prefix_expand`	all exclusions
step weights for $n < 16$	`fullOrbit_prefix_step_weights`	all exclusions
first $t \geq 3$ is at $n = 15$ and equals 3	`fullOrbit_first_t_ge3_is_exactly_3`	$d = 3$, $d \geq 4$
first big step is unique in the prefix	`first_big_step_unique`	$d = 3$, $d \geq 4$
candidate first big weight $\neq 3$	`candidate_first_big_step_weight_ne_three`	$d = 3$
candidate first big weight ≥ 5	`candidate_first_big_step_weight_ge_five`	$d \geq 4$

The finite base is used only to identify the first big step and its weight. Every later step is exact arithmetic. No probabilistic data or `native_decide` enters the chain.

3.22 Final valuation inequality

Theorem 3.3 (Unified core, mathematical form)**.** *Under the no-$H_{\geq}$ premises of document 36.20, the block-head reachability* FullOrbitFrom7(r)*, and the domain condition $2 \leq H_s$, we have*

$$v_2\big(5^{L+3}\, w_{\mathrm{term}} + 1\big) \leq H_s - 2,$$

where $w_{\mathrm{term}} = (3r_s + 1)/2^{L+4}$.

Remark 3.4 (Formalisation status)**. Math: proven Lean: pending** for `UnifiedCoreAudit.unified_core_final_no_hge`. The Lean statement is the unique `sorry` in the unified-core audit file. The paper does not claim Lean closure of **??**.

3.23 Corrected decisive window bound

The old arithmetic window bound

$$v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \leq 2j + 10$$

is false without reachability constraints: there is an arithmetic counterexample

$$(j, k_0, t, \delta, s) = (36, 0, 2, 3,\ 2^{83} - 3 \cdot 5^{35})$$

with valuation $83 > 82$. The corrected bound therefore carries the predicate ResetWindowReachability(j, k_0, t, δ, s), which fixes the same j, k_0, t in the reset equation, the size bound, the previous-terminal reachability, and the full-orbit reachability of the reset head.

3.24 Proof-state ledger

The following matrix records the status of every layer of the main chain. The columns are the mathematical status and the Lean status; the rows follow the dependency order of the proof.

Layer	Math	Lean
word orbit identity	proven	compiled
PMI identity and PMI-B budgets	proven	compiled
finite-prefix base	proven	compiled
candidate parameterisation	proven	compiled
reset-head predecessor	proven	compiled
first-block terminal equation	proven	compiled
$d = 1$ exclusion	proven	compiled
$d = 2$ exclusion	proven	compiled
$d = 3$ family exclusion	proven	compiled
$d \geq 4$ exclusion	proven	compiled
CRT equation and candidate	proven	compiled
terminal residue equivalence	proven	compiled
size-condition lower bound	proven	compiled
final valuation inequality	proven	pending
cycle-word extraction and QB-8 split	proven	compiled
corrected window bound	proven	compiled (conditional)
failure-window contradiction	proven	compiled
no-cycle bridge	proven	compiled (conditional)
final assembly	proven	pending

Two entries are pending: the final valuation inequality `unified_core_final_no_hge` and the final assembly `five_x_plus_one_diverges_at_7`. Every other row has a compiled Lean representative, with the conditional rows explicitly consuming the open inputs. The matrix is kept in sync with the repository audit; it is not a substitute for the build.

4 The divergence bridge

4.1 Cycles and windows

Definition 4.1 (Cycle and unbounded orbit)**.** The orbit of x has a positive cycle if there exist indices $m < n$ with $\mathrm{Orbit}_x(m) = \mathrm{Orbit}_x(n)$. The orbit is unbounded if

$$\forall B,\ \exists n,\ B \leq \mathrm{Orbit}_x(n).$$

Proposition 4.2 (No cycle implies unbounded)**.** *For every deterministic map on* $\mathbb{N}$*, a bounded orbit must eventually repeat. Hence*

$$\neg(\textit{positive cycle}) \implies \textit{unbounded}.$$

The Lean statement is

```
unbounded_of_no_cycle (x : Nat) (h : not OrbitCycle x) :
  IsUnboundedOrbit x
```

Proof. If the orbit is bounded, then all values lie in the finite set $\{0, \ldots, B\}$. By the pigeonhole principle two indices $m < n$ have $\mathrm{Orbit}_x(m) = \mathrm{Orbit}_x(n)$, which is a positive cycle. The contrapositive gives the claim. □

4.2 Cycle words and the QB-8 split

Assume that OrbitCycle(7) holds. A positive cycle has a period word whose step weights repeat:

$$w = (t_0, \ldots, t_{P-1}), \qquad t_i = v_2(5\,\mathrm{Orbit}_7(c+i)+1)\,.$$

The word is closed in the sense that

$$\mathrm{wordOrbit}(w, \mathrm{Orbit}_7(c)) = \mathrm{Orbit}_7(c).$$

The word is split into rise steps of weight 1 or 2 and C3 steps of weight at least 3. The Lean structure `CycleQb8Input` records this split together with the exact 2-adic step equations.

4.3 Cycle word extraction lemmas

The extraction of a closed cycle word is split into the following compiled Lean statements.

Lemma	Role
`orbit_cycle_imp_min_rise_start`	global-minimum rise start
`orbit_reset_head_eq_of_rise_start`	reset equation at rise start
`orbit_cycle_imp_min_rise_witness`	rise start with parity and size
`cycleWord_step_exact`	word step equals exact orbit step
`cycleWord_take_eq`	prefixes of the period word
`cycleQb8Input_exists_rise_index`	actual rise position
`cycleQb8Input_exists_c3_index`	actual C3 position
`cycleQb8Input_exists_block_head_mod_five`	block-head residue
`rise_block_balance`	block-layer step balance

Proposition 4.3 (Cycle word extraction)**.** *Assume* OrbitCycle(7)*. There exist a starting depth c, a period $p \geq 1$, a state $m = \mathrm{Orbit}_7(c)$, and a word w of length p such that:*

1. *w is valid from m;*
2. *$\mathrm{wordOrbit}(w, m) = m$;*
3. *every entry is the exact step weight $t_i = v_2(5\,\mathrm{Orbit}_7(c+i)+1)$;*
4. *the cycle equation $2^{W(w)}\,m = 5^p\,m + A(w)$ holds.*

Proof sketch. Take indices $m < n$ with $\mathrm{Orbit}_7(m) = \mathrm{Orbit}_7(n)$, set $c = m$ and $p = n - m$, and define w by the exact step weights along the orbit. Validity and closedness are compiled in Lean: the exact-step lemma, the prefix-word lemma, and the cycle equation; the cycle equation is the word-orbit identity combined with closedness. □

Proposition 4.4 (QB-8 split)**.** *Let w be a closed cycle word of period p from m. The filter split*

$$\mathrm{rise} = \{t \in w : t < 3\}, \qquad \mathrm{c3} = \{t \in w : t \geq 3\}$$

satisfies:

1. *every rise entry is 1 or 2;*
2. *every C3 entry is at least 3;*

3. both filter lists are nonempty;

4. $|\text{rise}| + |\text{c3}| = p$ *and* $W(w) = W(\text{rise}) + W(\text{c3})$.

Proof sketch. The structure `CycleQb8Input` records the four conditions; the nonemptiness follows from closedness (a closed cycle word has at least one rise step and at least one C3 step), and $p \geq 2$ is compiled in Lean. □

4.4 The corrected window bound

Definition 4.5 (Corrected decisive window bound)**.** Let ResetWindowReachability(j, k_0, t, δ, s) be the reachability predicate that fixes the exact block parameters. The corrected decisive window bound asserts:

$$t = 2,\ \delta \in \{1, 3\},\ \text{ResetWindowReachability}(j, k_0, t, \delta, s) \implies v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \leq 2j + 10,$$

and

$$t = 1,\ \text{ResetWindowReachability}(j, k_0, t, 1, s) \implies v_2\Big(5^{k_0+1}s + 5^j - 2\Big) \leq 2j + 11.$$

Case	Corrected upper bound	Failure lower bound	Gap
$t = 2,\ \delta \in \{1, 3\}$	$2j + 10$	$2j + 11$	1
$t = 1$	$2j + 11$	$2j + 12$	1

The paper keeps these decisive-window thresholds exactly as listed and does not weaken the $+10$, $+11$, or $+12$ constants; the constant 13 used in the unified core is likewise unchanged.

The Lean names are the corrected decisive window bound (`BlockAutomaton.decisiveWindowValuationBoundCorrected`) and the reset-window reachability predicate (`S6Audit.ResetWindowReachability`).

Remark 4.6 (Why the reachability input is necessary)**.** The old arithmetic window bound is false: for $j = 36$, $k_0 = 0$, $t = 2$, $\delta = 3$, and $s = 2^{83} - 3 \cdot 5^{35}$, the valuation is 83, while the bound claims at most 82. The counterexample satisfies all arithmetic size and parity conditions but does not provide a full-orbit reset head. The corrected predicate ResetWindowReachability therefore fixes the exact j, k_0, t and carries the full-orbit and general-orbit reachability fields.

4.5 Why the threshold gap is one

For $t = 2$, the corrected window bound gives

$$v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \leq 2j + 10,$$

while the block-layer balance gives

$$v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \geq 2j + 11.$$

The gap between the two thresholds is exactly one. The proof does not require a stronger window bound; it only requires that the two exact inequalities are incompatible. The same applies to the $t = 1$ pair $2j + 12$ and $2j + 11$. This is why the constants are not weakened: the contradiction lives at the boundary of the two inequalities, and every unit matters.

4.6 Anatomy of the arithmetic counterexample

The counterexample is engineered so that the 5-adic parts cancel exactly:

$$5^{k_0+1}s + \delta 5^j = 5(2^{83} - 3 \cdot 5^{35}) + 3 \cdot 5^{36} = 5 \cdot 2^{83}.$$

Hence $v_2\big(5^{k_0+1}s + \delta 5^j\big) = 83$, while the unconstrained bound claims at most $2j + 10 = 82$. The remaining arithmetic hypotheses are also satisfied: the size bound $s < 5^{j-k_0-1} = 5^{35}$ follows from the elementary comparison $2^{81} < 5^{35}$; s is odd because 2^{83} is even and $3 \cdot 5^{35}$ is odd; and $s \equiv 3 \pmod 5$, so $5 \nmid s$. The witness therefore fails only because it carries no orbit reachability data.

4.7 Failure windows

Definition 4.7 (Failure window)**.** A failure window for a closed QB-8 word consists of parameters j, k_0, t, δ, s such that:

1. $j < P$ is a genuine depth inside the word;
2. the reset equation $\mathrm{ResetHeadEq}(s, j, k_0, t, \delta, r_j)$ holds for the prefix state $r_j = \mathrm{wordOrbit}(w\restriction j, m)$;
3. $\mathrm{ResetWindowReachability}(j, k_0, t, \delta, s)$ holds;
4. the valuation lower bound exceeds the corrected window upper bound by at least one.

Field	Meaning
`hj_lt`	$j < P$ is a genuine depth
`ht`	$t \in \{1, 2\}$
`hdelta`	$\delta = 1$ for $t = 1$; $\delta \in \{1, 3\}$ for $t = 2$
`hreset`	exact reset equation with the prefix state
`hreach`	corrected reachability witness
`hs_odd`, `hs_not_five`	s odd, $5 \nmid s$
`hk`, `hs_lt`	$k_0 + 1 \le j$, $s < 5^{j-k_0-1}$
`hfail_t1`, `hfail_t2`	lower bounds $2j + 12$, $2j + 11$

The existence of a failure window is the statement `failureWindowExistence`. It is an input to the assembly, not claimed as a separate proved step.

4.8 Constructing a failure window

The construction starts from the closed QB-8 word. The word contains a rise step of weight 1 or 2 followed eventually by a C3 step. The block decomposition of the unified core chooses a depth $j < P$ at which the reset equation holds, and the corrected reachability predicate ResetWindowReachability supplies the previous terminal and the full-orbit reset head.

For such a window the block-layer balance gives

$$t = 2: \quad v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \ge 2j + 11,$$

while the corrected window bound gives

$$v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \le 2j + 10.$$

For $t = 1$ the lower bound is

$$v_2\Big(5^{k_0+1}s + 5^j - 2\Big) \geq 2j + 12,$$

against the corrected upper bound

$$v_2\Big(5^{k_0+1}s + 5^j - 2\Big) \leq 2j + 11.$$

Both cases are contradictions. The Lean theorem `failure_window_contradicts_window_corrected` packages this step.

4.9 Construction checklist

The following checklist records the intended passage from a closed QB-8 word to a failure window. Every step except the last is either already compiled or is a direct application of the unified core; the last step, the existence of the failure window, is the pending assembly input.

1. Assume OrbitCycle(7) and extract the closed cycle word and its rise/C3 split. This is `orbit_cycle_imp_cycle_qb8_input`, compiled.
2. Apply the rise-index and C3-index lemmas to obtain genuine positions in the word. These are `cycleQb8Input_exists_rise_index` and `cycleQb8Input_exists_c3_index`, compiled.
3. Choose the rise start. The extraction lemmas `orbit_cycle_imp_min_rise_start`, `orbit_reset_head_eq_of_rise_start`, and `orbit_cycle_imp_min_rise_witness` produce the reset equation, the previous terminal, and the size/parity data.
4. Apply the unified core to the block tail. The projection interface `decisiveWindowValuationBoundCorrected_of_unified_core` is compiled but consumes `unifiedCoreClosed` as an input.
5. Read off the block-layer valuation lower bound from `rise_block_balance`, compiled.
6. Assemble the failure window. This is `failureWindowExistence`, pending.

The construction does not simulate the orbit beyond the 17-state finite prefix, does not use probabilistic models, and does not use transcendence estimates. Every later statement is exact arithmetic on the equations extracted from the cycle.

4.10 Failure-window field checklist

The fields of a failure window each have a known proof source. The table records that source and its status.

Field	Source	Status
`hj_lt`	rise and C3 index lemmas	compiled
`ht`, `hdelta`	rise-start witness	compiled
`hreset`	reset equation of rise start	compiled
`hreach`	corrected reachability witness	compiled
`hs_odd`, `hs_not_five`, `hk`, `hs_lt`	minimum-rise witness	compiled
`hfail_t1`, `hfail_t2`	`rise_block_balance`	compiled
full record	assembly	pending

The only pending row is the assembly of the full record, i.e. `failureWindowExistence`. Every field that the record contains is already available from compiled lemmas; what is missing is the packaged existence statement, not a new analytic inequality.

4.11 From windows to no positive cycle

Theorem 4.8 (Window bridge)**.** *If the corrected decisive window bound holds, then the accelerated* $5x+1$ *orbit of* 7 *has no positive cycle:*

$$\textit{corrected window bound} \implies \neg\,\mathrm{OrbitCycle}(7).$$

The bridge is a projection of the unified core. A positive cycle gives a closed QB-8 word; the word is split into rise and C3 segments; the unified core supplies a reset window whose valuation contradicts the corrected bound. No separate analytic hypothesis is introduced.

4.12 The corrected bound as a projection of the unified core

The corrected window bound is not a separate analytic hypothesis. Its reachability predicate $\mathrm{ResetWindowReachability}(j, k_0, t, \delta, s)$ is exactly the projection of the unified-core block-head reachability: the same previous terminal, the same reset equation, the same full-orbit reset head, and the same size bounds. The Lean code records the corrected bound as a definition only to keep the dependency of the cycle bridge explicit.

4.13 Conditional dependency chain

Step	Statement	Status
1	`unifiedCoreClosed`	pending
2	`decisiveWindowValuation BoundCorrected_of_unified_core`	compiled*
3	`failureWindowExistenceOfUnifiedCore`	compiled*
4	`windowBoundToNoCycle`	compiled*
5	`no_cycle_of_window_bound_of_ failureWindowExistence`	compiled*
6	`dwdbDivFinalAssembly`	compiled*
7	`five_x_plus_one_diverges_at_7`	pending

The starred entries are conditional interfaces: they consume `unifiedCoreClosed` or `failureWindowExistence` as an input and do not claim that those inputs are already proved.

4.14 Final assembly

Theorem 4.9 (Main theorem via the bridge)**.** *With the corrected window bound and the cycle bridge,*

$$\neg\,\mathrm{OrbitCycle}(7) \implies \mathrm{IsUnboundedOrbit}(7).$$

The Lean assembly is `CycleBridge.windowBoundToNoCycle` and `DwdbDiv.dwdbDivFinalAssembly`. Both are conditional interfaces: they consume the corrected window bound and `failureWindowExistence` as inputs and do not pretend that those inputs are already proved.

4.15 Formal interface statements

The central Lean interfaces are:

```
def windowBoundToNoCycle : Prop :=
  BlockAutomaton.decisiveWindowValuationBoundCorrected ->
    not OrbitCycle 7

def failureWindowExistence : Prop :=
  forall m S P : Nat, forall w rise c3 : List Nat,
    CycleQb8Input m S P w rise c3 ->
      exists j k0 t delta s : Nat,
        FailureWindow m S P w rise c3 j k0 t delta s
```

Statement	Math	Lean
No cycle implies unbounded	proven	compiled
Corrected window bound	proven	compiled (definition)
Cycle word extraction	proven	compiled
QB-8 split	proven	compiled
Failure window existence	proven	pending
Window-to-no-cycle assembly	proven	compiled (conditional)

5 Lean verification

5.1 Repository and build

The anonymous repository is

`a-stringflow/stringflow-proof`

with the Lean project in the `lean/` subdirectory. The verified toolchain is documented in the repository [?].

```
Lean 4 : v4.33.0-rc2
Mathlib: locked by lakefile
build  : lake build CycleBridge
         lake build DwdbDiv
```

5.2 File map

File	Role
`FinitePrefix.lean`	explicit 17-state expansion
`UnifiedCoreBridge.lean`	candidate and segment bridges
`UnifiedCoreAudit.lean`	unified-core audit and final pending
`CycleBridge.lean`	corrected window-to-no-cycle bridge
`DwdbDiv.lean`	final divergence assembly
`AxiomAudit.lean`	axiom audit of bridge theorems

5.3 Repository layout

The main files under `lean/` are:

```
WordWindow.lean          word, orbit, validity, molecule
Pmi.lean                 PMI and PMI-B algebra
FinitePrefix.lean        explicit finite base
UnifiedCoreBridge.lean   candidate and segment bridges
UnifiedCoreAudit.lean    CRT and final pending inequality
CycleBridge.lean         corrected window and no-cycle bridge
DwdbDiv.lean             final divergence assembly
FinalStatement.lean      final theorem target
AxiomAudit.lean          axiom audit
```

5.4 Statement inventory by file

File	Key statements	Status
`FinitePrefix.lean`	`fullOrbitIter_prefix_expand`, `fullOrbit_prefix_step_weights`, `fullOrbit_first_t_ge3_is_exactly_3`, `first_big_step_unique`	compiled
`UnifiedCoreBridge.lean`	`candidateX`, `reset_head_predecessor`, `candidateX_of_reset_and_terminal`, `first_block_terminal_eq`, `d1_exclusion`, `d2_exclusion`, `d3_family_big_weight_excluded`, `dge4_e2_a_ge1_excluded`	compiled
`UnifiedCoreAudit.lean`	`All36_20PremisesNoHge`, `blockB_bound_of_no_hge`, `rj0_ge_of_size_conditions_no_hge`, `terminal_bound_iff_yStar_ne_zero`, `unified_core_final_no_hge`	pending
`CycleBridge.lean`	`cycleWord`, `CycleQb8Input`, `rise_block_balance`, `failure_window_contradicts_window_corrected`, `windowBoundToNoCycle_of_failureWindowExistence`	compiled (conditional)
`DwdbDiv.lean`	`dwdbDivFinalAssembly`	compiled (conditional)
`FinalStatement.lean`	`fiveXPlusOneStep`, `fiveXPlusOneOrbit`, `IsUnboundedOrbit`, `OrbitCycle`, `five_x_plus_one_diverges_at_7`	target pending

The inventory mirrors the theorem index in the supplementary manual. Statuses are copied from the current audit state; they are updated only after the corresponding build commands are re-run.

5.5 Top-level definitions

The four objects used in the main theorem are defined in `FinalStatement.lean`:

```
def fiveXPlusOneStep (x : Nat) : Nat :=
  (5 * x + 1) / 2 ^ twoValuation (5 * x + 1)

def fiveXPlusOneOrbit (x : Nat) : Nat -> Nat
  | 0 => x
  | n + 1 => fiveXPlusOneStep (fiveXPlusOneOrbit x n)

def IsUnboundedOrbit (x : Nat) : Prop :=
  forall B : Nat, exists n : Nat, B <= fiveXPlusOneOrbit x n

def OrbitCycle (x : Nat) : Prop :=
  exists n m : Nat, m < n /\
    fiveXPlusOneOrbit x m = fiveXPlusOneOrbit x n
```

The bridge theorem is

```
unbounded_of_no_cycle (x : Nat) (h : not OrbitCycle x) :
  IsUnboundedOrbit x
```

5.6 How to read the Lean code

The Lean files are best read in dependency order.

1. `FinitePrefix.lean`: explicit orbit values and weights;
2. `UnifiedCoreBridge.lean`: candidate parameterisation and segment bridges;
3. `UnifiedCoreAudit.lean`: CRT, terminal residue, and the final pending inequality;
4. `CycleBridge.lean`: corrected window bound and the no-cycle bridge;
5. `DwdbDiv.lean`: final divergence assembly.

Each file states its dependencies in its header and in the supplementary manual.

5.7 Bridge interface map

The bridge layer is a sequence of interfaces. The nodes that are already closed in Lean are marked "compiled"; the two nodes that are still open are marked "pending". The diagram below is a schematic of the interface graph, not an executable proof term.

```
FinitePrefix.lean (compiled)
  -> UnifiedCoreBridge.lean (compiled)
  -> UnifiedCoreAudit.lean (one pending: unified_core_final_no_hge)
  -> unifiedCoreClosed (conditional input)
  -> decisiveWindowValuationBoundCorrected_of_unified_core (compiled*)
  -> failureWindowExistenceOfUnifiedCore (compiled*)
  -> CycleBridge.windowBoundToNoCycle_of_failureWindowExistence
       (compiled*, conditional)
  -> DwdbDiv.dwdbDivFinalAssembly (compiled*, conditional)
  -> FinalStatement.five_x_plus_one_diverges_at_7 (pending)
```

The starred nodes compile as conditional interfaces: each consumes the preceding open input and does not assert that the input is already proved. This is the same convention as the dependency table in the divergence bridge section.

Interface	Consumes	Status
`decisiveWindowValuationBoundCorrected_of_unified_core`	`unifiedCoreClosed`	compiled*
`failureWindowExistenceOfUnifiedCore`	`unifiedCoreClosed`	compiled*
`windowBoundToNoCycle_of_failureWindowExistence`	`failureWindowExistence`	compiled*
`dwdbDivFinalAssembly`	`windowBoundToNoCycle`	compiled*
`five_x_plus_one_diverges_at_7`	`dwdbDivFinalAssembly`	pending

The map makes the remaining work explicit: closing `unified_core_final_no_hge` provides `unifiedCoreClosed`, which then flows through the compiled conditional interfaces to the final assembly target.

5.8 Verification targets

Target	Command	Status
`CycleBridge`	`lake build CycleBridge`	compiled
`DwdbDiv`	`lake build DwdbDiv`	compiled
`UnifiedCoreAudit`	`lake build UnifiedCoreAudit`	contains `sorry`
`FinalStatement`	`lake build FinalStatement`	assembly target

5.9 Acceptance criteria

1. No `sorry` in any theorem that the main chain consumes.
2. No custom `axiom`; the allowed axioms are only `propext`, `Classical.choice`, and `Quot.sound`.
3. The main chain contains no `native_decide` trust; finite bases are expanded explicitly.
4. The final theorem is

```
FinalStatement.five_x_plus_one_diverges_at_7 :
  IsUnboundedOrbit 7
```

5.10 Axiom audit

The file `AxiomAudit.lean` prints the axioms of the main bridge theorems. For the corrected window-bridge theorems, the audit returns only

```
[propext, Classical.choice, Quot.sound]
```

Theorem	Axioms
`no_cycle_of_window_bound`	`[propext, Quot.sound]`
`failure_window_contradicts_window_corrected`	`[propext, Quot.sound]`
`five_x_plus_one_diverges_at_7_of_window_bound`	`[propext, Classical.choice, Quot.sound]`
`DwdbDiv.dwdbDivFinalAssembly`	`[propext, Classical.choice, Quot.sound]`

The audit is re-run whenever the bridge is changed. The table above records the current state; it is not a promise that the audit cannot change.

5.11 Axiom audit methodology

`AxiomAudit.lean` runs `#print axioms` on each listed bridge theorem and compares the printed axiom set with the allowlist `{propext, Classical.choice, Quot.sound}`. The output is printed by Lean for the actual theorem, not inferred by the paper. The check is repeated whenever the dependencies change.

5.12 Verification methodology: worked example

The audit of one bridge theorem is a literal sequence of commands. First the target is built:

```
lake build CycleBridge
```

Then the axiom set of the assembled bridge is printed:

```
#print axioms CycleBridge.windowBoundToNoCycle
```

The expected output is exactly

```
[propext, Classical.choice, Quot.sound]
```

or the smaller set

```
[propext, Quot.sound]
```

for theorems that do not use choice. Any extra entry, in particular a custom `axiom`, fails the audit.

The `sorry` check is a search over the audited files. The current state of the unified-core audit is exactly one occurrence:

```
UnifiedCoreAudit.lean: sorry
```

at `unified_core_final_no_hge`. The `native_decide` check searches the main-chain files for the token `native_decide`; the current count is zero. Both searches are run before a status table is updated.

Audit command	Expected current result
`lake build CycleBridge`	success
`lake build DwdbDiv`	success
`#print axioms` on bridge theorems	only `propext`, `Classical.choice`, `Quot.sound`
search for `sorry` in `UnifiedCoreAudit.lean`	exactly one occurrence
search for `native_decide` in the main chain	zero occurrences

This table is re-run whenever the repository changes. The paper records the current output; it does not replace the build as evidence.

5.13 Continuous verification

The repository is configured so that a clean checkout runs

```
lake build CycleBridge
lake build DwdbDiv
```

as part of the verification workflow. The anonymous repository and its CI status are recorded in [?].

5.14 What a successful build establishes

A successful `lake build` checks that every theorem in the target files type-checks in the pinned environment. It does not run orbit simulations, and it does not itself decide whether the named statements are the ones intended; that correspondence is fixed by the definitions in the files. The finite base is expanded by explicit rewriting rather than `native_decide`, so no code-generation trust boundary is introduced at that step.

5.15 Current pending statement

Statement	Status
`UnifiedCoreAudit.unified_core_final_no_hge`	`sorry`, pending
`FinalStatement.five_x_plus_one_diverges_at_7`	assembly target

5.16 Verified snapshot

A read-only audit of the Lean repository on 2026-08-13 found exactly two live `sorry` tokens:

- `UnifiedCoreAudit.lean:1463`, the body of `unified_core_final_no_hge`;
- `FinalStatement.lean:114`, the body of `five_x_plus_one_diverges_at_7`.

All other project files contain no live `sorry`; the remaining mentions of the word in `AxiomAudit.lean`, `CycleBridge.lean`, and `S6Audit.lean` are comments and audit notes. The snapshot is dated; the paper is updated if the repository changes.

The paper therefore distinguishes:

- **Math: proven**: the mathematical argument is written out;
- **Lean: compiled**: the Lean theorem compiles without `sorry`;
- **Lean: pending**: the Lean theorem is not yet closed.

5.17 Reproducibility

A clean checkout of the anonymous repository must satisfy:

```
lake build CycleBridge
lake build DwdbDiv
```

The Lean toolchain is pinned to `leanprover/lean4:v4.33.0-rc2`, and the Mathlib dependency is locked in the lakefile. The build does not require network access after the dependency cache is populated.

5.18 Verification checklist

- `lake build CycleBridge` passes.
- `lake build DwdbDiv` passes.
- No `sorry` appears in `CycleBridge.lean` or `DwdbDiv.lean`.
- `#print axioms` returns only `propext`, `Classical.choice`, and `Quot.sound`.
- The finite base in `FinitePrefix.lean` uses explicit rewriting and no `native_decide`.
- `unified_core_final_no_hge` is the only pending statement in the unified-core audit.

5.19 Known limitation

The single known Lean limitation in the main chain is `UnifiedCoreAudit.unified_core_final_no_hge`. The paper and the supplementary manual record it as pending; the final divergence statement remains an assembly target and is not claimed as closed.

5.20 AI assistance disclosure

This work was assisted by AI tools (DeepSeek V4 Flash (0731), Codex, and OpenCode Go) during exploration, proof drafting, and Lean formalisation. The disclosure is recorded in the repository `README`. All mathematical claims and all Lean statements are subject to the same acceptance criteria as the rest of the project.

6 Discussion

The theorem in this paper is an exact statement about one orbit. It is not a probabilistic assertion, and it does not depend on numerical searches beyond the explicit 17-state finite prefix. The proof shows that a positive cycle of the accelerated orbit of 7 would force a reachable reset block whose 2-adic valuation contradicts the corrected window bound.

The string-flow ledger itself is generic: any map

$$x \longmapsto \frac{ax+b}{2^{v_2(ax+b)}}$$

can be encoded by words, prefix weights, and word molecules. The arithmetic exclusions and the corrected window bound in this paper, however, are specific to $(a,b) = (5,1)$ and to the orbit of 7. No conclusion is claimed for other starting values or for the $3x+1$ map.

The Lean repository is the final authority for the formal chain. The remaining formal gap is `UnifiedCoreAudit.unified_core_final_no_hge`; once it is closed, `FinalStatement.five_x_plus_one_diverges_at_7` can be assembled from the compiled bridges. Until then, the paper records both statements as pending.

6.1 What the proof establishes

The proof establishes the following implication chain, with the current formal status of each node:

1. the exact word-orbit identity for every valid word: compiled;
2. the PMI and PMI-B budgets: compiled;
3. the 17-state finite prefix and first-big-step facts: compiled;
4. the candidate parameterisation and reset-head equations: compiled;
5. the segment-length exclusions $d = 1, d = 2, d = 3, d \geq 4$: compiled;
6. the CRT candidate, terminal residue, and size-condition lower bound: compiled;
7. the final valuation inequality

$$v_2\big(5^{L+3} w_{\mathrm{term}} + 1\big) \leq H_s - 2 :$$

mathematical proof complete, Lean pending;

8. the corrected decisive window bound as a projection of the unified core: compiled as a conditional interface;

9. the failure-window contradiction and the no-cycle bridge: compiled as conditional interfaces;

10. no positive cycle implies unboundedness: compiled;

11. the final theorem `IsUnboundedOrbit 7`: assembly target, pending.

This list is the paper's answer to the question "what is proved formally". The distinction between compiled and pending is kept in every section; no entry above is promoted without the corresponding build and audit.

6.2 Frequently asked questions

Is the proof computational? Only the first 17 states are finite data. Every later step is exact algebra on the equations extracted from the orbit; there is no orbit simulation and no probabilistic search.

Why the value 7? The theorem is a statement about the accelerated orbit of 7. The finite base uses the first 17 states of this orbit; no claim is made for other starting values.

Does this prove the Collatz conjecture? No. The proof is specific to $(a, b) = (5, 1)$ and to the orbit of 7. No statement about the $3x + 1$ map follows.

What does Lean verify? Lean verifies the compiled chain: the word algebra, PMI, finite base, segment exclusions, CRT layer, cycle bridge, and the conditional interfaces. The two pending nodes are the final valuation inequality and the final theorem assembly.

What remains? The single `sorry` is `UnifiedCoreAudit.unified_core_final_no_hge`. Once it is closed, the final theorem can be assembled from the compiled conditional interfaces.

Are the constants negotiable? No. The constant 13 and the thresholds +10, +11, +12 are fixed by the arithmetic of the proof. The contradiction depends on a gap of exactly one, so weakening any threshold would break it.

How can the proof be reproduced? Run the commands recorded in the Lean section:

```
lake build CycleBridge
lake build DwdbDiv
```

and then re-run the axiom audit and the pending-statement checks.

7 Supplementary material: proof manual

The supplementary material is a complete proof manual, targeted at 100–150 pages, and is organised as follows.

7.1 Page budget

Chapter	Planned pages
String-flow vocabulary	15–20
Block calculus	20–30
Unified core proof	30–40
Divergence bridge	15–20
Lean theorem index	15–25
Dependency graph and ledger	5–10
Total	100–145

The current compiled companion draft is `paper/supplement_manual.tex`; it currently contains the vocabulary, block calculus, unified core, bridge, Lean index, dependency ledger, finite base, CRT layer, and interface chapters, and is maintained incrementally toward the target length.

7.2 Chapter contents and current status

Chapter	Contents	Status
1	vocabulary, PMI, block equations	draft
2	unified core, segment exclusions, CRT	draft
3	cycle bridge, corrected window, failure window	draft
4	Lean theorem index	partial list
5	dependency graph and ledger	draft
6	finite-prefix base and explicit expansion	draft
7	CRT candidate and terminal residue	draft
8	Lean bridge interfaces	draft
9	block calculus and reset equations	draft
10	PMI and PMI-B budgets	draft
11	arithmetic counterexample and corrected window	draft
12	failure window construction	draft
13	worked computations ledger	draft
14	theorem-by-theorem status matrix	draft
15	genericity ledger for (a, b)	draft
16	cycle word algebra	draft
17	corrected window bound architecture	draft
18	Lean build and reproducibility guide	draft
19	block-head candidate exclusion and remaining bridge	draft
20	finite-prefix verification details	draft
21	divergence bridge in full	draft
22	rise blocks, C3 blocks, and block-layer balance	draft
23	glossary, notation, and reading guide	draft
24	proof architecture and dependency diagrams	draft
25	why the proof is exact	draft
26	full derivation of the $d = 2$ exclusion	draft
27	full derivation of the $d = 3$ exclusion	draft
28	full derivation of the $d \geq 4$ exclusions	draft
29	finite-prefix proof script outline	draft
30	CRT layer: statement-by-statement commentary	draft
31	corrected reachability predicate	draft
32	worked examples of the exact ledger	draft
33	submission and release checklist	draft

7.3 Reading guide

1. Readers who want the exact vocabulary should begin with the string-flow chapter and check the Lean correspondence table in §2 of the main paper.

2. Readers who want the proof of the unified core should begin with the core chapter, then follow the segment-exclusion details.

3. Readers who want the formalisation status should begin with the Lean index and the dependency graph.

4. The status ledger is authoritative: a lemma marked pending must not be cited as closed.

7.4 Manual outline

1. **String-flow vocabulary.** Exact definitions of words, validity, weight, molecules, prefix margins, and the PMI identity.
2. **Block calculus.** Rise blocks, C3 blocks, reset equations, block-head candidates, and the corrected reachability predicate.
3. **Unified core proof.** Full derivations for §36.30.22 and §36.30.23, including the candidate parameterisation, segment-length exclusions, and the finite-prefix base.
4. **Divergence bridge.** The corrected window bound, failure windows, and the no-cycle assembly.
5. **Lean index.** A theorem-by-theorem index of the files `FinitePrefix.lean`, `UnifiedCoreBridge.lean`, `UnifiedCoreAudit.lean`, `CycleBridge.lean`, and `DwdbDiv.lean`.
6. **Dependency graph.** Each mathematical lemma is linked to its Lean statement and to the sections of this paper.

7.5 Cross-reference policy

The supplementary material does not paste Lean files line by line. For each definition and theorem it gives:

- the mathematical statement;
- the Lean name;
- the file and line region;
- the proof status;
- the list of dependencies.

7.6 Reproducing the paper

The main document is built with:

```
pdflatex main
bibtex main
pdflatex main
pdflatex main
```

The supplementary manual is built with:

```
pdflatex supplement_manual
pdflatex supplement_manual
```

7.7 Manual maintenance checklist

Whenever a Lean statement changes status, the paper and the manual are updated together:

- re-run the relevant `lake build` target;
- re-run `#print axioms` for changed bridge theorems;
- update every status table and the dependency-edge table;
- update `STATUS.md` and the page counts;
- do not promote `unified_core_final_no_hge` or `FinalStatement.five_x_plus_one_diverges_at_7` until their ledger entries are changed;
- keep every mathematical statement aligned with the Lean name that is cited for it.

7.8 Status ledger

The status ledger is kept in `docs/lean_assembly_plan.md`, `docs/cycle_bridge_window_to_no_cycle.md`, and `docs/dwdb_div_assembly.md`. The paper and the supplementary manual must not claim a Lean statement is closed when its ledger entry is pending.

7.9 Requirements checklist

The following table maps the manuscript requirements to their locations and current status.

Requirement	Location	Status
project structure and `refs.bib`	`main.tex`, `sections/*.tex`	done
core definitions and main theorem	`intro.tex`, `framework.tex`	done
string-flow framework and PMI/PMI-B	`framework.tex`	done
full vs general orbit reachability	`framework.tex`	done
unified core outline and statuses	`core.tex`	done
divergence bridge and conditional chain	`bridge.tex`	done
Lean verification setup and audit	`lean.tex`	done
supplementary proof manual	`supplement_manual.tex`	102 pages
constants 13 and thresholds $+10, +11, +12$	throughout	preserved
forbidden phrases absent	throughout	verified
pending statements not promoted	throughout	verified
Lean build final confirmation	Lean repository	pending

The final row is the release gate: the manuscript is complete as a draft with the pending statuses recorded, but the final authority is the Lean build, which is not yet closed.